

CLIMB: CURRICULUM LEARNING FOR MULTI-AGENT BEHAVIOR TREE GENERATION WITH SMALL LANGUAGE MODELS

Yichen Wang¹, Zhongxuan Cai^{*1}, YINUO Liu¹, Yuhao Wang¹, Tianjian Jiang¹, Yuanxi Peng¹

¹College of Computer Science and Technology,

National University of Defense Technology, Changsha, China

{wangyichen.wyc, caizhongxuan, liuyinuo0103, wangyuhao24, jiangtianjian, pyx}@nudt.edu.cn

ABSTRACT

Behavior trees have become a popular approach for modeling robot behaviors through recursive tick signals, but their automated construction remains challenging. Recent work explores using large language models (LLMs) to generate behavior trees from natural-language task descriptions. However, in multi-agent coordination scenarios, LLMs face critical challenges: they struggle to learn coordination primitives, are too large for practical deployment, and lack spatial cognition, causing task failures. Addressing these challenges, we present CLIMB (Curriculum Learning for Multi-agent Behavior Tree Generation with Small Language Models): at the supervision stage, we leverage symbolic planning algorithms to construct high-quality training data encoding correct multi-agent cooperation priors; we further optimize both training and inference stages—at the training stage, we design a curriculum learning strategy that identifies coordination complexity from both difficulty and utility perspectives to progressively improve learning effectiveness; at the inference stage, we provide the model with spatial cognition to prevent generating spatially unreachable behaviors. We validate our approach through both numerical simulation and physical 3D simulation in ROS 2/Gazebo, demonstrating that our method significantly improves execution success rate compared to standard supervised fine-tuning, and our fine-tuned 1B small language model outperforms current mainstream LLMs while reducing token cost.

Index Terms— Behavior Trees, Multi-Agent Coordination, Curriculum Learning, Constrained Decoding, Execution Evaluation

1. INTRODUCTION

Behavior trees (BTs) have become the popular way to organize complex robot behavior in modern robotics [1, 2]. Compared with finite-state machines, their modular, hierarchical structure unifies reactive control, sequential tasks, and fault recovery, driving adoption in commercial platforms, game AI, and multi-robot systems [3].

Automating BT construction has traditionally relied on symbolic planning [4, 5], evolutionary algorithms [6, 7], or reinforcement learning [8], each of which needs a hand-crafted domain model. Large language models (LLMs) promise a more general route [9, 10]: generating BTs directly from natural-language task descriptions using pretrained world knowledge.

However, as we demonstrate in this paper, multi-agent coordination poses significant challenges for LLMs, making it extremely difficult to generate correct behavior trees and accomplish direct multi-agent collaboration tasks. The failures stem from two key issues:

LLMs struggle to learn multi-agent coordination primitives, and they lack spatial cognition of actual environments.

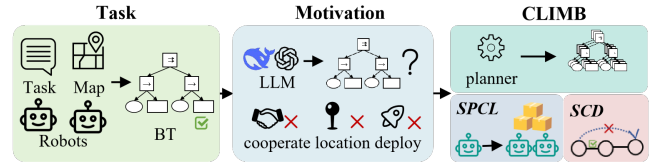


Fig. 1. CLIMB enables a small language model to generate multi-agent behavior trees.

We propose **CLIMB (Curriculum Learning for Multi-agent Behavior Tree Generation with Small Language Models)** (Fig. 1), which grounds supervision, training, and inference in symbolic planning: executor-validated data supplies correct cooperation priors, a curriculum orders examples by coordination complexity, and constrained decoding grounds generation in the platform map. A 1B model trained and constrained this way surpasses 5-shot LLMs at one-tenth the inference cost. The main contributions are:

- **A planner-grounded multi-agent behavior tree dataset.** We construct executor-validated mission-tree pairs through symbolic planning and behavior tree expansion, covering complex coordination tasks that LLM-generated data cannot reliably solve.
- **Structure-Paced Curriculum Learning (SPCL).** We design a curriculum that orders training examples by difficulty and utility, improving the model’s learning of multi-agent coordination primitives.
- **Symbolic Constrained Decoding (SCD).** We enhance the model’s spatial cognition at inference.
- **Comprehensive validation.** We evaluate CLIMB in both numerical simulation and physics-engine-driven 3D virtual simulation, showing that a 1B model outperforms mainstream LLMs while reducing token cost.

2. RELATED WORK

LLM behavior-tree generation. LLM behavior-tree generation. BTGenBot [9] fine-tunes lightweight LLMs on language-BT pairs; BTGenBot-2 [11] improves efficiency. Other works prompt general LLMs for direct generation [10], tree expansion [12], and multi-robot dialogue [13], but outputs may parse yet fail during execution. MRBTP [14] keeps a symbolic planner as core generator and uses the LLM only for action pre-planning. Our work instead validates

^{*}The corresponding author. Contact Email: caizhongxuan@nudt.edu.cn

every training pair with a planner and executor and trains a small model with curriculum learning and constrained decoding.

LLM-based multi-agent collaboration. Existing LLM-based systems coordinate embodied agents through messages, deliberation, or centralized planning [15, 16, 17]. Their coordination relies on runtime messages or LLM/planner calls, keeping communication and inference on the critical path, while multi-agent reinforcement learning instead learns teammate policies [18]. Behavior trees instead execute through a periodic tick propagated from the root [2]. Unlike these approaches, our single generated tree encodes cooperation in its topology and joint-action nodes, so execution needs only the recursive tick signal, with no LLM inference in the loop.

Curriculum learning and data selection. Classical curricula order examples from easy to hard [19]. LLM-era methods extend this to different data levels: Skill-it [20] orders and mixes data by prerequisite skills, DELIFT [21] selects instruction examples by estimated utility, and Token Cleaning [22] scores individual tokens before supervised fine-tuning. DUCL [23] shows that difficulty alone is insufficient and jointly scores utility as well, motivating SPCL’s joint difficulty–utility design for multi-agent BT generation.

3. PRELIMINARIES AND PROBLEM FORMULATION

3.1. Preliminaries

A behavior tree is a rooted hierarchical policy for robot behavior [2]. Internal nodes define control flow:  Sequence ticks children left-to-right and succeeds only if all children succeed,  Fallback succeeds if any child succeeds, and  Parallel ticks multiple children concurrently. Leaves are either  Conditions, which test predicates and return Success or Failure, or  Actions, which execute robot commands and may return Running until completion (Fig. 2(a)).

A behavior tree is executed through a periodic tick: each control period, the root sends a tick, and every ticked node returns Running, Success, or Failure to its parent, which determines the next child to tick. This recursive tick–status propagation is a compact discrete-time control signal. For a team, one tree uses Parallel branches to coordinate per-agent subtrees, while joint-action and synchronization nodes carry explicit agent and object identifiers. Cooperation is thus encoded in the tree topology and exercised by ticks alone (Fig. 2(b)).

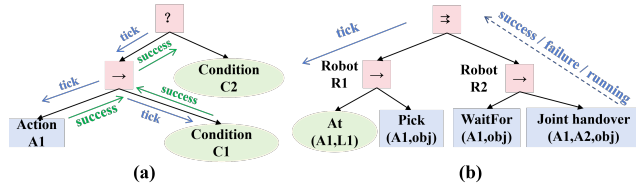


Fig. 2. Behavior-tree preliminaries. (a) BT structure and tick–status signal flow. (b) Two-robot relay with a joint action.

3.2. Problem Formulation

Let a mission be $x = (x_{\text{usr}}, x_{\text{sys}})$, where x_{usr} is a natural-language instruction and x_{sys} describes the platform. From x_{sys} we extract a platform map $\mathcal{M} = (\mathcal{R}, \mathcal{I}, \mathcal{L}, \mathcal{E}, \mathcal{C})$, where $\mathcal{R}$, $\mathcal{I}$, $\mathcal{L}$, $\mathcal{E}$, and $\mathcal{C}$ denote the sets of robots, objects, locations, connectivity edges, and charging stations, respectively. Our goal is to learn a small generator

Algorithm 1: Forward-Search Multi-Agent Dataset Construction

Input: Domain $\mathcal{D}$; target size N ; seed r .

Output: Validated dataset $\mathcal{S}$.

```

1  $\mathcal{S} \leftarrow \emptyset$ ;
2 while  $|\mathcal{S}| < N$  do
3    $\tau \leftarrow \text{SampleTask}(\mathcal{D}, r)$ ;
4    $\pi \leftarrow \text{ForwardSearch}(\text{NominalDomain}(\tau))$ ;
5    $\tau' \leftarrow \text{InjectFaults}(\tau)$ ;
6   if  $\neg \text{ValidPlan}(\pi, \tau')$  then
7     continue;
8    $y \leftarrow \text{CompileBT}(\pi, \tau')$ ;
9   if  $\text{Executor}(y, \tau')$  then
10     $\mathcal{S} \leftarrow \mathcal{S} \cup \{(\text{Paraphrase}(\tau'), y)\}$ ;
11 return  $\mathcal{S}$ 

```

g_θ , parameterized by θ , that maps $(x, \mathcal{M})$ to an executable behavior tree y coordinating the team through per-agent subtrees and joint-action nodes; the locations, robots, and objects referenced by y must exist in the target platform.

A symbolic executor $\Phi(y; x) \in \{0, 1\}$ returns 1 iff y is a syntactically correct behavior tree, executes without errors on $\mathcal{M}$, and reaches all mission goals. On a held-out test set $\mathcal{T}$, success rate is

$$\text{SR}(\theta) = \frac{1}{|\mathcal{T}|} \sum_{x \in \mathcal{T}} \Phi(g_\theta(x, \mathcal{M}); x). \quad (1)$$

Given a training set $\mathcal{D}$ of paired missions and target behavior trees, we train g_θ with the standard teacher-forced objective

$$\mathcal{L}(\theta) = -\mathbb{E}_{(x,y) \sim \mathcal{D}} \sum_t \log p_\theta(y_t \mid x, y_{<t}, \mathcal{M}), \quad (2)$$

where t indexes token positions in y , $y_{<t}$ is the prefix before position t , and p_θ is the model distribution. Our goal is to improve the success rate of the generated behavior trees in executing the mission.

4. METHOD

4.1. Dataset

Each sampled task τ (Algorithm 1, Fig. 3) specifies a platform description x_{sys} , an initial state, a goal, and a connectivity graph, from which we extract the platform map $\mathcal{M}$. A symbolic forward search on a nominal domain without recovery actions yields a plan π ; after fault injection, π is replayed on the full domain and compiled into a behavior tree y with parallel branches, rendezvous nodes for joint actions, and fallback branches for recovery. Only trees passing the symbolic executor Φ are retained and paired with a paraphrased instruction x_{usr} , giving training pairs (x, y) whose targets are executor-validated by construction.

4.2. Structure-Paced Curriculum Learning

In general, fine-tuning presents pairs in a fixed order. Structure-Paced Curriculum Learning (SPCL) performs curriculum learning from two perspectives: difficulty and utility.

Difficulty. We measure difficulty from two views: a semantic view using the model’s negative log-likelihood and top-1 tag error, and a structural view using tree-level counts (number of robots,

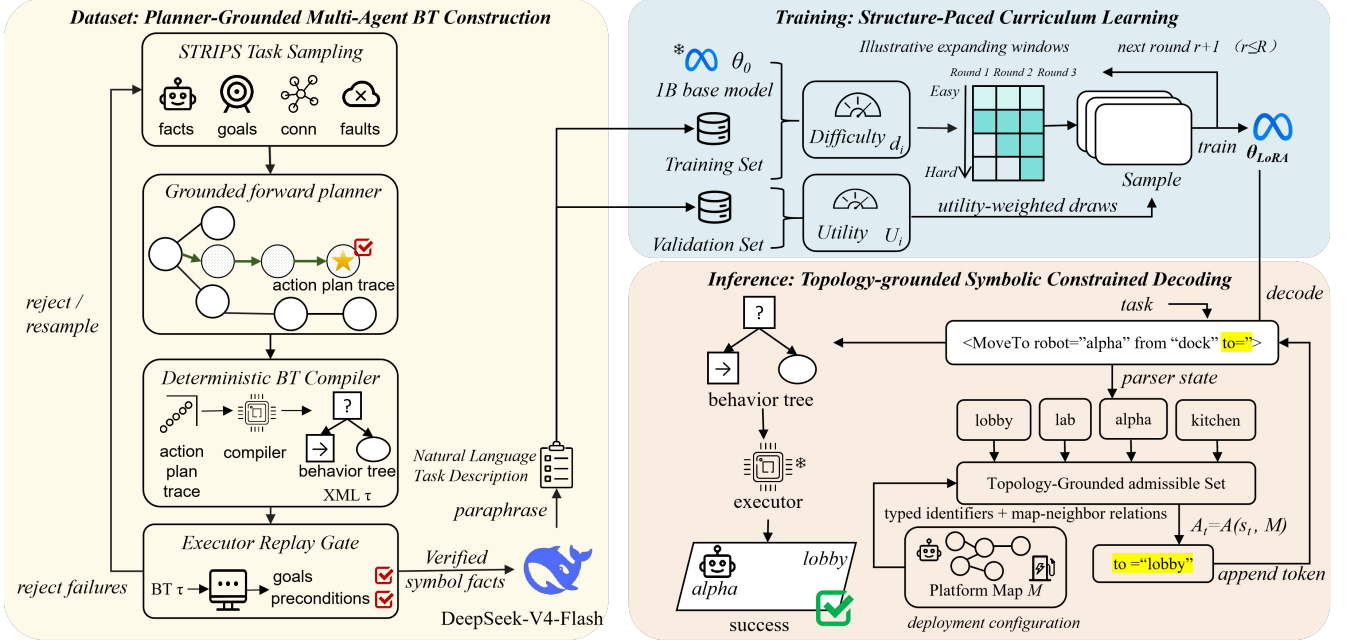


Fig. 3. Overview of CLIMB: planner-grounded dataset construction (supervision), SPCL (training), and SCD (inference).

faults, synchronization nodes, joint-action nodes, fallback nodes, tree depth, and total nodes). Let Z be the feature matrix in one view, with row Z_i and entry Z_{ij} . We fuse its p columns as

$$\text{Fuse}(Z_i; Z) = \sum_{j=1}^p \alpha_j \rho(Z_{ij}), \quad \alpha_j = \frac{|v_j|}{\sum_{\ell=1}^p |v_\ell|}, \quad (3)$$

where p is the number of features in the view, ρ is mid-rank normalization to $[0, 1]$, and $v = (v_1, \dots, v_p)^\top$ is the leading eigenvector of the rank-normalized covariance. Eigenvector weighting down-weights redundant features and reduces to equal averaging when features are uncorrelated; we fuse within each view and across views to obtain $d_i \in [0, 1]$.

Utility. We bias sampling toward examples close to the validation set. For view $v \in \{\text{semantic (text), structural (tree)}\}$ and validation set $\mathcal{V}$, we compute a Gaussian kernel similarity to the k nearest validation neighbors,

$$u_i^{(v)} = \frac{1}{k} \sum_{j \in \mathcal{N}_k(i, \mathcal{V})} \exp\left(-\delta_{ij}^{(v)2} / 2\sigma_v^2\right), \quad (4)$$

where $\mathcal{N}_k(i, \mathcal{V})$ is the set of k nearest validation examples to training sample i , $\delta_{ij}^{(v)}$ is the embedding distance between i and neighbor j in view v , and σ_v is the median of all positive distances from training samples to their nearest validation neighbors in view v . The two view scores are mid-rank normalized and fused into U_i using the same PCA-loading rule as for difficulty.

Schedule. We sort examples by d_i into K equal-mass buckets and, across stages $s = 1, \dots, R$, sample from an expanding window $w_s = \min(K, \lceil Ks/R \rceil)$, drawing with probability proportional to $U_i m_{s,b_i}$, where b_i is the bucket of example i and $m_{s,b}$ is the multiplier for bucket b at stage s . The multipliers are uniform in stage 1, $m_{s,b} \propto 1/\bar{U}_b$ in intermediate stages, and $m_{R,b} \propto \bar{D}_b/\bar{U}_b$

in the final stage, with $\bar{D}_b, \bar{U}_b$ the per-bucket means of d_i, U_i . The per-bucket target proportions are rounded to multiples of $1/K$ and lower-bounded by $1/K$.

4.3. Symbolic Constrained Decoding

At inference time, we enhance the model’s spatial cognition by grounding generation in the current platform map. This prevents the model from producing locations that do not exist or spatial transitions that are invalid in the target environment.

Let s_t be the parser state after prefix $y_{<t}$. The parser knows which spatial field is expected next, such as a move source, a move target, or a location argument. Given the platform map $\mathcal{M} = (\mathcal{R}, \mathcal{I}, \mathcal{L}, \mathcal{E}, \mathcal{C})$, SCD constructs the admissible set $A_t = A(s_t, \mathcal{M})$ by keeping only location identifiers that exist in $\mathcal{L} \cup \mathcal{C}$, and by allowing a move $a \rightarrow b$ only when $(a, b) \in \mathcal{E}$. The constrained distribution is

$$\tilde{p}_\theta(y_t | y_{<t}, x, \mathcal{M}) = \text{softmax}(\ell_t - \infty \cdot \mathbf{1}[\cdot \notin A_t]), \quad (5)$$

where ℓ_t are the logits at step t . Tokens outside A_t receive zero probability, so the generated locations and moves are consistent with the actual spatial layout of the environment.

5. EXPERIMENT

5.1. Experimental setup

Tasks and scenarios. Given a natural-language mission and the platform map $\mathcal{M}$ for a team of 2–3 robots, the system must output one executable behavior tree. Training and validation use four seen domains (warehouse, hospital, search–rescue, office), while testing uses only two unseen domains: library and greenhouse. The main evaluation is a 480-task coordination test set, consisting of 320 relay-transport and 160 joint-heavy-transport missions; 357 tasks contain

injected recoverable faults. We also validate CLIMB on 38 coordination missions in 3D ROS 2/Gazebo simulations with 2–3 TurtleBot3 robots.

Verification and metrics. In numerical simulation, a generated behavior tree is counted as successful only if it is syntactically valid, parses, and ticks to *SUCCESS*. We report execution success rate and average token cost per task on the 480 held-out tasks. In the 3D robotic tasks, we measure task completion rate.

Baselines. All trained systems use Llama-3.2-1B-Instruct as the base model. BTGenBot-MA adapts the BTGenBot [9] supervised fine-tuning recipe to our format and serves as the training-side reference. Four general-purpose LLMs—GPT-5.6-Luna, Kimi-K2.5, DeepSeek-V4-Flash, and Qwen3.5-Plus—are prompted with one or five demonstrations. We also validate CLIMB on a comparable-size small model, DeepSeek-R1-Distill-Qwen-1.5B. As a symbolic reference, we run MRBTP [14], which does not consume natural language: it receives the oracle formalization of each instance (initial predicates, goals, topology, reachability, action schemas), and its returned per-robot BTs are executed in the same symbolic executor as all methods, without any reference plan or target tree.

5.2. Results

Numerical simulation. We compare CLIMB with baselines on multi-agent tasks. Despite oracle symbolic input, MRBTP lags clearly: its fallbacks cannot recover from faults requiring actions missing from its model, and its large backup trees stall on tightly coupled three-robot tasks. CLIMB achieves the highest execution success rate, outperforming all four LLMs even under 5-shot prompting and consistently improving over BTGenBot-MA on both small language models, while keeping token consumption far below prompted LLMs and roughly on par with BTGenBot-MA (Table 1).

Table 1. Comparison with baselines on multi-agent BT generation.

Model	SR (%)		Tokens	
	1-shot	5-shot	1-shot	5-shot
GPT-5.6-Luna	6.46	50.42	2,678	6,366
Kimi-K2.5	8.12	54.38	1,813	5,981
DeepSeek-V4-Flash	10.83	59.17	1,799	9,024
Qwen3.5-Plus	20.62	60.63	6,628	10,686
BTGenBot-MA[9]				
Llama3.2-1B-Instruct	59.0		1,102	
DeepSeek-R1-Distill-Qwen-1.5B	38.8		1,076	
MRBTP-Adapted [14]	42.1		–	
CLIMB (<i>ours</i>)				
Llama3.2-1B-Instruct	67.9		1,104	
DeepSeek-R1-Distill-Qwen-1.5B	62.9		1,092	

3D simulation. CLIMB completes the most missions and matches or exceeds all four 5-shot LLMs at substantially lower token cost; all executor-verified trees also complete in this 3D setting, confirming the numerical gains transfer to practical execution (Table 2, Fig. 4).

5.3. Ablation Study

Table 3 removes one component at a time. For the supervision ablation, DeepSeek regenerates each target under the same 5-shot

Table 2. ROS 2/Gazebo task completion.

System	Complete	SR (%)
GPT-5.6-Luna (5-shot)	19/38	50.0
Kimi-K2.5 (5-shot)	24/38	63.2
DeepSeek-V4-Flash (5-shot)	20/38	52.6
Qwen3.5-Plus (5-shot)	21/38	55.3
BTGenBot-MA[9]	19/38	50.0
CLIMB(ours)	27/38	71.1

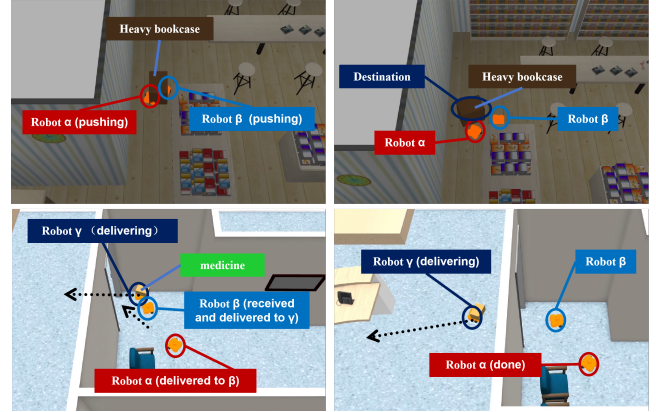


Fig. 4. ROS 2/Gazebo task completion under CLIMB.

prompting and executor filter until failures stop resolving, yielding 4,160 verified targets at 69.3% valid yield; SPCL then re-derives its curriculum on them. This swap causes the largest drop on both backbones while adding 49.6M generation tokens on top of an otherwise identical inference cost of about 0.5M, whereas removing SPCL or SCD costs far less at unchanged token cost. Every component contributes more on the weaker backbone, so CLIMB is most valuable when the base model is weakest.

Table 3. Component ablation of CLIMB. ([†]Planner-free targets from DeepSeek).

Configuration	Llama-3.2-1B		R1-Distill-1.5B	
	SR (%)	Tokens	SR (%)	Tokens
w/o planner superv. [†]	49.0	1,090	51.3	1,081
w/o SPCL	63.8	1,107	50.6	1,094
w/o SCD	64.2	1,098	51.3	1,075
CLIMB (ours)	67.9	1,104	62.9	1,092

6. CONCLUSION

We presented CLIMB, which converts natural-language multi-agent collaboration descriptions into executable behavior trees. It combines a symbolic-planning-based, executor-validated dataset, SPCL curriculum learning over difficulty and utility, and SCD spatial grounding at inference. Evaluations in numerical and 3D simulations show strong performance, outperforming state-of-the-art LLMs at substantially lower cost. Future work will extend CLIMB to larger heterogeneous teams, long-horizon tasks, and real robots.

7. ACKNOWLEDGEMENT

This work is supported by Innovation Research Foundation of National University of Defense Technology (ZK26-15).

8. REFERENCES

- [1] Michele Colledanchise and Petter Ögren, *Behavior Trees in Robotics and AI: An Introduction*, CRC Press, 2018.
- [2] Matteo Iovino, Edvard Scutkins, Jonathan Styrud, et al., “A survey of behavior trees in robotics and ai,” *Robotics and Autonomous Systems*, vol. 154, pp. 104096, 2022.
- [3] Petter Ögren, “Behavior trees in robot control systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, pp. 81–107, 2022.
- [4] Michele Colledanchise, Diogo Almeida, and Petter Ögren, “Towards blended reactive planning and acting using behavior trees,” *arXiv preprint arXiv:1611.00230*, 2016.
- [5] Zhongxuan Cai, Minglong Li, Wanrong Huang, and Wenjing Yang, “BT expansion: A sound and complete algorithm for behavior planning of intelligent robots with behavior trees,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, vol. 35.
- [6] Pei Zhang, Joe Collenette, Jakob Dahlberg Larsson, et al., “Learning behavior trees for autonomous agents with hybrid constraints evolution,” *Applied Sciences*, vol. 8, no. 7, pp. 1077, 2018.
- [7] Michele Colledanchise, Ramviyas Parasuraman, and Petter Ögren, “Learning of behavior trees for autonomous agents,” *IEEE Transactions on Games*, pp. 183–189, 2019.
- [8] Jonathan Styrud, Victor Karlsson, Alexander Lidström, et al., “Combining planning and learning of behavior trees for robotic assembly,” in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 11511–11517.
- [9] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci, “Btgenbot: Behavior tree generation for robotic tasks with lightweight llms,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 9684–9690.
- [10] Jiong Ao, Fan Wu, Yansong Wu, et al., “Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1233–1239.
- [11] Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo Matteucci, “Btgenbot-2: Efficient behavior tree generation with small language models,” *arXiv preprint arXiv:2602.01870*, 2026.
- [12] Jonathan Styrud, Matteo Iovino, Mikael Norrlöf, Petter Ögren, and Christian Smith, “Automatic behavior tree expansion with llms for robotic manipulation,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [13] Artem Lykov, Maria Dronova, Naglov Nikolay, et al., “Llm-mars: Large language model for behavior tree generation and nlp-enhanced dialogue in multi-agent robot systems,” *arXiv preprint arXiv:2312.09348*, 2023.
- [14] Yishuai Cai, Xinglin Chen, Zhongxuan Cai, et al., “Mrbtp: Efficient multi-robot behavior tree planning and collaboration,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, Oral presentation. arXiv:2502.18072.
- [15] Hongxin Zhang, Weihua Du, Jiaming Shan, et al., “Building cooperative embodied agents modularly with large language models,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [16] Zhao Mandi, Shreya Jain, and Shuran Song, “Roco: Dialectic multi-robot collaboration with large language models,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 286–299.
- [17] Shyam Sundar Kannan, Vishnunandan L. N. Venkatesh, and Byung-Cheol Min, “Smart-llm: Smart multi-agent robot task planning using large language models,” in *IEEE/RSJ IROS*, 2024, pp. 12140–12147.
- [18] Yinuo Liu, Zhongxuan Cai, Tianjian Jiang, et al., “T-camel: Teammate-causal-aware multi-agent learning,” in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026.
- [19] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, et al., “Curriculum learning,” in *International Conference on Machine Learning (ICML)*, 2009.
- [20] Mayee F. Chen, Nicholas Roberts, Kush Bhatia, et al., “Skill-it! a data-driven skills framework for understanding and training language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, pp. 36000–36040.
- [21] Ishika Agarwal, Krishnateja Killamsetty, Lucian Popa, et al., “Delift: Data efficient language model instruction fine-tuning,” in *International Conference on Learning Representations (ICLR)*, 2025, pp. 100488–100509.
- [22] Jing Pang, Nan Di, Zinan Zhu, et al., “Token cleaning: Fine-grained data selection for llm supervised fine-tuning,” in *International Conference on Machine Learning (ICML)*, 2025.
- [23] Zishang Jiang, Jinyi Han, Tingyun Li, et al., “Difficulty is not enough: Curriculum learning for llms fine-tuning must consider utility,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.